Project Blueprint: Tabular RL and DQN

> **Executive Summary: Decoding the Professor's Intent**
> This project is not primarily a coding test; it is an exercise in *experimental methodology and scientific defense*. By explicitly allowing generative AI but heavily weighting the oral defense (50% of the grade), the professor is evaluating your ability to architect fair experiments, diagnose algorithmic failures, and deeply understand the mechanics of RL. The code is merely a vehicle for the 6–8 page analytical report and the in-person defense.

1. Strategic Overview

To secure full marks, teams must shift focus from simply "making the agent win" to rigorously proving *why* it wins or loses. The evaluation criteria demand reproducible configurations (fixed seeds), statistically significant reporting (confidence bands over multiple seeds), and ablation studies that isolate the impact of specific neural network components.

- **The Defense Rule:** You must be able to explain every line of code, including AI-generated segments. Prepare for live modifications (e.g., altering the ε-decay rate or changing the bin boundaries for discretization).
- **The Metric Rule:** Never compare algorithms on "episodes." Always map learning curves to "environment steps," as tabular RL and DQN consume data at vastly different episodic rates.
- **The Baseline Rule:** Negative results are valid. A DQN that fails to solve MountainCar due to sparse rewards will earn full marks if the failure is properly diagnosed and documented.

2. Theoretical Analysis (Analysis-Only Component)

The assignment requires a specific written analysis regarding the limitations of Tabular RL and DQN in complex environments. This forms the theoretical backbone of the report.

The Curse of Dimensionality in Continuous Spaces

Environments like BipedalWalker and MuJoCo locomotion tasks (HalfCheetah, Ant) feature continuous, multi-dimensional action spaces. For example, HalfCheetah has a 6-dimensional continuous action space. To apply DQN—which requires discrete actions—one must discretize this space. If we discretize each dimension into just 5 bins, the total action space becomes $5^6 = 15{,}625$ discrete actions. The DQN output layer would require 15,625 nodes, making the maximization step over actions computationally expensive and exploration hopelessly inefficient.

Tabular RL suffers similarly from observation spaces. If a 17-dimensional continuous state space is discretized into 10 bins per dimension, the Q-table requires 10^{17} entries, vastly exceeding memory limits and requiring impossible amounts of data to visit each state-action pair.

Algorithm Families for Continuous Control

For MuJoCo and BipedalWalker, Actor-Critic methods designed for continuous spaces are required [1]. Suitable algorithms include:

- **PPO (Proximal Policy Optimization):** A robust on-policy algorithm that outputs the parameters (mean and standard deviation) of a continuous probability distribution (like a Gaussian) from which actions are sampled.
- **SAC (Soft Actor-Critic):** An off-policy method that maximizes a trade-off between expected return and entropy, promoting excellent exploration in continuous domains.
- **TD3 (Twin Delayed DDPG):** A deterministic off-policy algorithm that directly outputs a continuous action vector, mitigating the overestimation bias inherent in standard Q-learning.

The Failure of Tabular Methods on Pixels

Atari environments process visual pixels (e.g., 210 × 160 RGB). Even if downsampled to an 84 × 84 grayscale grid, a single frame contains 7,056 pixels. Since a pixel takes 256 possible intensity values, the number of distinct states is 256^{7056}. Tabular methods rely on exact state visitation; the probability of visiting the exact same pixel configuration twice is functionally zero, preventing any value updates from generalizing [2].

3. Implementation Guidelines: Tier 1 & 2

Tier 1: Tabular vs. DQN Comparison

The goal is to contrast the sample efficiency and stability of a simple Q-table against a neural network. Choose **CartPole-v1** for a standard control problem, or **MountainCar-v0** if you want to write a compelling discussion on sparse rewards.

Algorithm	Key Challenges & Mechanics	Required Ablations / Studies
Tabular (Q-Learning)	Requires state discretization. The update rule: `Q(s,a) ← Q(s,a) + α[R + γ max Q(s',a') − Q(s,a)]`	Vary the number of bins (e.g., 6, 10, 20). Justify the ranges (e.g., clipping velocity to a practical maximum).
DQN	Requires Replay Buffer for I.I.D data, and a Target Network to stabilize the moving target `R + γ max` Q_{target}`(s',a')`.	Monitor for catastrophic forgetting. Track the effect of ε-greedy decay rates on early vs late performance.

Professor's Trap: Do not evaluate algorithms solely at the end of training. Evaluate greedy performance (ε = 0) every N steps (e.g., 5,000 steps) during training, freezing the weights. Plot these evaluation returns separately from training returns.

Tier 2: Advanced DQN & Ablations

LunarLander-v3 is the recommended environment. It features a larger state space (8D) and requires careful balancing of fuel expenditure and crash penalties.

The required ablation study is critical. You must remove key components of DQN and document the collapse:

- **No Target Network:** Update the Q-values using the same network that is being optimized. This causes the target to shift constantly, usually resulting in a divergent loss and collapsing return curves.
- **No Replay Buffer (Online DQN):** Train on sequential transitions as they occur. This breaks the independent and identically distributed (i.i.d) assumption of neural networks, leading to severe instability.

4. Experimental Rigor and Reporting

Your 6–8 page report must resemble a formal machine learning conference paper. Visualizing results properly is where most points are won or lost.

- **Plotting Learning Curves:** Run at least 3 (preferably 5) distinct random seeds for every configuration. In matplotlib, use `plt.fill_between` to shade the standard deviation (or 95% CI) around the mean curve.
- **Equitable X-Axes:** Standardize the x-axis to "Cumulative Environment Steps". Tabular methods might finish an episode in 15 steps early on, while DQN takes 200. Comparing them by "Episode" makes Tabular look artificially slower.
- **Hardware and Wall-clock:** Neural networks are sample-efficient but computationally expensive. Tabular methods are sample-inefficient but computationally cheap. Document the total CPU hours per run to highlight this trade-off.

> **Handling Negative Results:** If your DQN fails on MountainCar, explain why. MountainCar issues a -1 reward every step until the flag is reached. Unless the random ε-policy accidentally hits the flag, the replay buffer fills only with -1s, and the network learns that all states are equally bad. Contrast this with Tabular RL's optimistic initialization.

5. Surviving the Oral Defense

The oral defense constitutes 50% of the grade. Both team members must be prepared to defend the entire codebase. The generative AI policy explicitly states that inability to explain AI-assisted code will result in severe penalties.

Anticipated Live Modifications & Questions:

1. *"Change the discretization boundaries for the pole angle in CartPole. What happens?"* (Answer: If bins are too coarse around the vertical center, the agent cannot distinguish slight tilts and will fail to balance. If bins are too fine, it requires too much data to populate the Q-table).
2. *"Explain the gather() operation in your PyTorch DQN loss function."* (Answer: It extracts the Q-values corresponding to the specific actions taken in the batch of experiences, rather than the maximum Q-values which are only used for the next-state targets).
3. *"How did you implement the target network update?"* (Answer: Either via a hard copy every `target_update_freq` steps using `load_state_dict()`, or via a soft polyak update $\theta_{target} = \tau\theta + (1-\tau)\theta_{target}$).

6. Structure of the Final Repository

Ensure reproducibility. The professor must be able to run a single command (e.g., `python experiments/run_tier1.py`) and see your graphs generate perfectly. Use a fixed seed strategy across `numpy.random`, `torch.manual_seed`, and `env.reset(seed=...)`.

```
project/
  agents/
    tabular.py        # Contains TabularAgent class
    dqn.py            # Contains DQNAgent, QNetwork, ReplayBuffer classes
  envs/
    discretizers.py   # State space binning logic
  experiments/
    tier1_cartpole.py # Config and loop for Tier 1
    tier2_ablation.py # Config and loop for Tier 2 ablations
  results/            # Saved weights (.pt), logs, and plots
  report/             # LaTeX or Word source for the PDF
  README.md           # Exact execution commands for reproducibility
```

Expanded Annotated Bibliography

- **Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.**
  Relevance to Project: This is the definitive textbook for tabular methods. Chapter 6 details the theoretical differences between on-policy SARSA and off-policy Q-learning. Chapter 9 is critical for Tier 1, as it discusses state-space discretization (including coarse and tile coding), providing the mathematical foundation for why mapping continuous values to discrete bins allows tabular methods to converge despite the curse of dimensionality.
- **Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.**
  Relevance to Project: The seminal paper that introduced the modern Deep Q-Network (DQN) architecture. Your Tier 1 and Tier 2 implementations directly rely on the two primary innovations introduced here to stabilize neural network training: 1) **Experience Replay**, which breaks the temporal correlation of sequential observations and smooths data distribution, and 2) **Target Networks**, which provide a temporarily fixed optimization target to prevent the network from chasing its own updates. Your Tier 2 ablation study will directly test the necessity of these two mechanisms.
- **van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with Double Q-learning. *AAAI*.**
  Relevance to Project (Tier 3B): Standard DQN suffers from severe overestimation bias because the `max` operator uses the same noisy values to both select and evaluate an action. This paper demonstrates how to decouple them by using the online network to select the optimal action, and the target network to evaluate its expected return. Essential reading if you are implementing Double DQN.
- **Wang, Z., Schaul, T., Hessel, M., et al. (2016). Dueling network architectures for deep reinforcement learning. *ICML*.**
  Relevance to Project (Tier 3B): Introduces the Dueling architecture, which splits the DQN hidden layers into two separate streams: one estimating the state Value function *V(s)* and another estimating the Advantage function *A(s, a)*. This is particularly useful in environments like LunarLander where, in certain states, the exact action taken has little effect on the outcome, allowing the network to learn state values much more efficiently.
- **Software & Framework References:**
  - *Gymnasium (Farama Foundation):* The modern standard API for RL environments (replacing OpenAI Gym). Essential for querying exact observation and action space bounds (e.g., `env.action_space.n`).
  - *CleanRL (Huang et al., 2022):* Recommended as a code reference for DQN due to its single-file, highly readable implementation style, making it much easier to study the raw math than heavily abstracted libraries.
  - *Stable-Baselines3 (Raffin et al., 2021):* A reliable production-level PyTorch implementation. The syllabus allows this strictly as a performance benchmark to validate if your custom DQN is learning at the expected rate and achieving the threshold scores.